Seminar 2

Object-Oriented Design, IV1350

Mo Wang mowang@ug.kth.se

2026-03-30

Project Members:
[Mo Wang, mowang@ug.kth.se]

Declaration:

By submitting this assignment, it is hereby declared that all group members listed above have contributed to the solution. It is also declared that all project members fully understand all parts of the final solution and can explain it upon request.

It is furthermore declared that no part of the solution has been copied from any other source (except for resources presented in the Canvas page for the course IV1350), and that no part of the solution has been provided by someone not listed as a project member above.

1. Introduction

This report focuses on the **Design phase** of the `Repair Electric Bike` system and includes the design method, interaction diagrams, the resulting class diagram, and an evaluation of the design against the assessment criteria. The system design is based on the given requirements specification with both normal and alternative workflow scenarios, as well as following the `Domain Model` and `System Sequence Diagram` established earlier, the details of which can be found in Appendices A, B and C.

1.1. Requirements of Seminar 2

The requirements of Seminar 2 are to design a complete, well-structured object-oriented architecture for the `Repair Electric Bike` system. The seminar focuses on applying established principles from the course literature, particularily in Section 4.1 and Chapter 5 of `A First Course in Object-Oriented Development`.

The design must satisfy the following requirements:

- **Layered architecture following the MVC and Layer patterns**. The program must be organised into appropriate layers with correct direction of dependencies and no mixing of responsibilities.
- **High cohesion, low coupling, and strong encapsulation**. The design shall demonstrate a well-defined public interface, clear responsibilities, and minimal dependencies between components.
- **Complete coverage of the entire Repair Electric Bike scenario**. All basic and alternative flows must be designed, including the startup sequence (the `main` method).
- **Interaction diagrams for all behaviour**. The Result chapter must include sequence or communication diagrams covering *every* system operation, including alternative flows and the startup sequence.
- **A class diagram consistent with the interaction diagrams**. The diagram must include all classes used in the design, but may omit excessive detail for clarity.

Implementation of the user interface and data persistence layer is out of scope. A simplified `View` class and a minimal integration layer are therefore used.

This report therefore describes the design method, the resulting class and interaction diagrams, as well as a discussion evaluating the design according to the required principles.

2. Method

Based on the domain model (DM) and system sequence diagram (SSD) from analysis, which can be found in Appendix, the system design can be carried out by first dividing the system into the layered architecture pattern based on MVC and therefore designing one system operation at a time while aiming high cohesion, low coupling and high degree of encapsulation.

2.1. Step 1. Use the patterns MVC and layer

The first step involves usage of layered architectural MVC pattern, which includes several layers: view, controller, model, integration, data and startup, which can be visualized in Figure 1 showing a package diagram. It shows the different layers, where each of them has a clear responsibility and only interacts with the layer beneath it, following the direction of the dependencies illustrated in Layer pattern.

View layer concerns interactions with user while core logic is handled in model layer. Controller layer acts as a bridge between the view and the model and carries the necessary system operations. By calling each system operation, the equivalent operations in the model and integration are handled. The integration layer acts as a middle ground between the core logic and data layer, which in turn is responsible for data storage.

Finally, the startup layer handles program initialization of the rest five layers and serves as the main function.

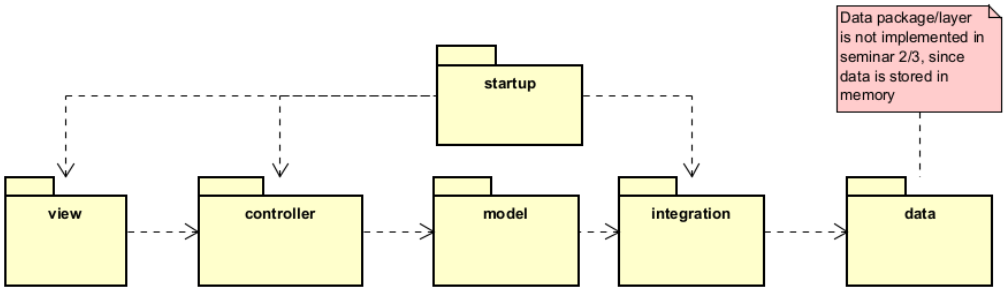

Figure 1: The layered MVC architecture pattern is divided in layers with each package corresponding to each layer

The `bike repair system` implements the MVC pattern by dividing the entities in domain model as classes into different layers. Both View and Controller layer introduces its own classes, such as `ReceptionistController` and `TechnicianController` in Controller for different authorization roles, as well as `ControllerCreator` that centralizes controller class constructions.

2.2. Step 2. Design one system operation at a time

During the design process of the system, each system operation is analyzed into its corresponding interaction diagram. These diagrams illustrates the flow of execution and the order of execution in an enumerated sequence and provides visualization on how messages are passed between objects, unlike class diagrams which primarily visualizes static system relationships.

In this case, communication diagrams are used since they offer more compact view, instead of sequence diagrams because sequence diagrams occupy large amount of space and become complex when tracing the flow of messages across different class instances. On the other hand, communication diagrams ephasizes the relationships between objects and exchanged messages.

Designing each sequence diagram for each system operation begins by creating the necessary class instances as well as the main instance from View class to a Controller class. etc.

A typical system operation begins with a call from the View to the Controller, which then delegates work to Model and Integration objects following the message flow in the SSD. For instance, the `searchCustomer` operation can be visualized as a sequence diagram by tracing the method calls across layers: starting from the `View` class calling

`searchCustomer` on the `ReceptionistController`, which in turn interacts with the relevant classes in the Model and Integration layers to look up the customer data.

The communication diagrams for each system operation are drawn and visualized using Astah tool, which is specialized for UML diagram design. The tool helps to design a structured and standardized UML representation of interactions as well as easier modification of diagrams and improved consistency across the system design process.

2.3. Step 3. High cohesion, low coupling, and a high degree of encapsulation with a small, well-defined public interface

High cohesion, low coupling, and a high degree of encapsulation with a small, well-defined public interface are several important OOD guidelines.

When a new functionality is added, method should only be created that these principles are satisfied as fully as possible. For instance, a class or a method should be responsible for its dedicated and solemn functionality with as few connections to its neighbour classes as possible.

For achieving high degree of encapsulation, some internal helper functions that works inside a class should be hidden if it doesn't contribute to connection with other classes. As an example, helper methods such as the phone-format conversion inside `TelephoneNumber` are kept private, because no class outside that type needs access.

Additionally, each operation is placed in the class that represents its abstraction with the necessary data for the operation. The domain model can be used to identify potential new classes that may need to be introduced.

Objects are favoured over primitive data types as well as avoiding static members, since primitive data and static members are not object oriented which causes harder to achieve these three guidelines. For instance, `Cost` is modeled as its own class instead of using primitive numbers, since cost include amount, currency and probably future metadata.

2.4. Step 4. Class diagram for relationship between classes

After designing system operation sequence diagrams and considering the three OOD guidelines, class diagram for relationship can be drawn by adding all classes that has been used in sequence diagrams and add them into the class diagram. Each class is placed into its corresponding package, where each package maps into a layer, with lines connecting the class relationships.

The class diagram is also drawn using Astah tool, since it provides efficient support for organizing classes into packages, visualizing relationships clearly, and maintaining a consistent UML design.

2.5. Step 5. Implement the new design in code

In the last step, the new design in code is implemented, with the help from class diagram and sequence diagrams for each operation. The class diagram helps to create the skeleton code with each class name, whereas the system operation helps to fill out the relevant

implementation steps of each system operation. The design is implemented in code to verify that the interactions and dependencies defined in the diagrams are feasibly designed.

By implementing the design in code, the code editor VSCode is used since it offers syntax highlightening and variable autocompletion. For compiling and testing the code, Maven is used for full codebase compilation, handle project dependencies and automate build tasks as well as handling testing using JUnit test cases.

3. Result

Following the method's steps, sequence diagrams for each system operation and class diagrams are created. These diagrams serve as blueprints for the subsequent implementation.

The system adopts a layered architecture based on the MVC pattern, where each layer is mapped to a corresponding package. Since the project includes internal computational logic, database or file operations, and an interactive user interface, the program is divided into layers that each handle a specific concern. This separation allows responsibilities to be clearly partitioned - for example, the data layer manages file operations, while the model layer handles internal computation.

Every system operation defined in the SSD is converted into a communication diagram, presented in the same order as in the SSD. This chapter therefore describes the design of every system operation, including the interaction diagrams and class diagram.

3.1. The `searchCustomer` system operation

The first operation, `searchCustomer`, retrieves a customer using a telephone number as input. In the searchCustomer case, the `View` class calls the method `searchCustomer` with a phone number in free format on `ReceptionistController` can be first written. Since a customer object needs to be returned, the receptionist controller class should find the corresponding customer. This can be done by calling `CustomerRegistry` that stores and handles customer informations, the function `find` is called by passing a phone number in canonical E164 format. In this case, the given phone must be converted from any format into canonical E164 format through `TelephoneNumber` class, before handing it into CustomerRegistry. The full process of is visualized in Figure 2.

In the `searchCustomer` operation several important design principles are applied, including encapsulation with a small, well-defined public interface, high cohesion and low coupling. First, the telephone number provided by the view is accepted in any arbitrary format, but must be normalized by converting into canonical E164 form in the method `toE164` in `TelephoneNumber` class. This ensures that data does not "appear out of nothing" and that both original and transformed values are explicit written.

Both `CustomerDTO` and `BikeDTO` are immutable DTO classes located in the integration layer. This prevents the view from receiving references to mutable class objects and ensures strong encapsulation. Using the same object name throughout the interaction

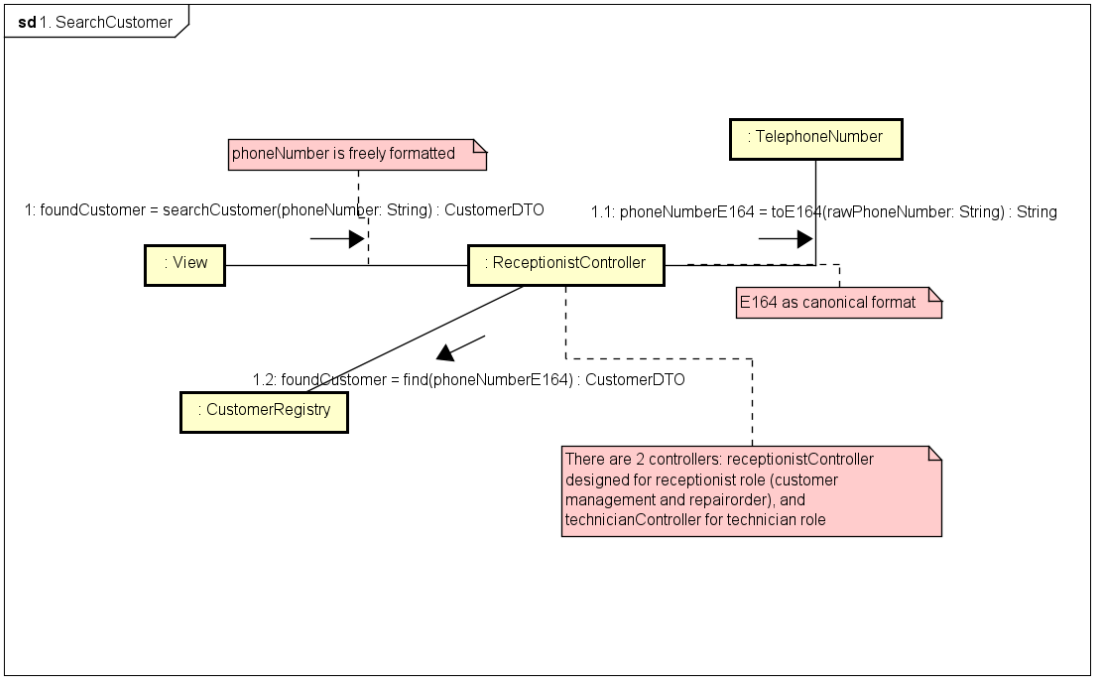

Figure 2: A communication diagram for `searchCustomer` system operation

diagram makes it clear that the same instance is passed across layers, which simplifies understanding of the data flow.

Cohesion is increased since each class performs only the task it is intended for: `TelephoneNumber` formats numbers, `CustomerRegistry` retrieves customer information, and the controller coordinates flow without performing logic. Coupling is minimized by ensuring that the controller does not take responsibility in customer storage, nor how telephone numbers are validated.

3.2. The `createRepairOrder` system operation

The `createRepairOrder` system operation can be translated by identifying the operation steps. The operation starts with searching the broken bike from bike serial number and creating a `ProblemDTO` with description and the found broken bike. Afterwards, the `ProblemDTO` is passed to `ReceptionistController` which first fetch its corresponding customer and therefore creates a `RepairOrder` which is saved later in `RepairOrderRegistry`. These steps are visualized in Figure 3.

The `createRepairOrder` operation builds on the DTO-policy established in the design. `ProblemDTO` is used as input because it represents a snapshot of customer problem data that may expand in future (e.g. urgency or additional flags). Using a DTO instead of several primitive parameters avoids long parameter lists and preserves low coupling

and a stable public interface.

`RepairOrder` is a mutable entity in the model layer. Its state evolves as diagnostics and tasks are added later in the workflow. Meanwhile, `CustomerDTO` and `BikeDTO` are immutable because customer and bike data do not change during program execution. This clear separation between entity and DTO avoids ambiguity and prevents leaking modifiable internal state to external layers.

Encapsulation is maintained by ensuring that the controller does not directly modify the internals of a `RepairOrder`. Instead, it delegates to the model, which contains the logic for constructing and validating a new repair order. Coupling is reduced because only `RepairOrderRegistry` handles data persistence, while `ReceptionistController` only forwards the ProblemDTO and orchestrates the operation.

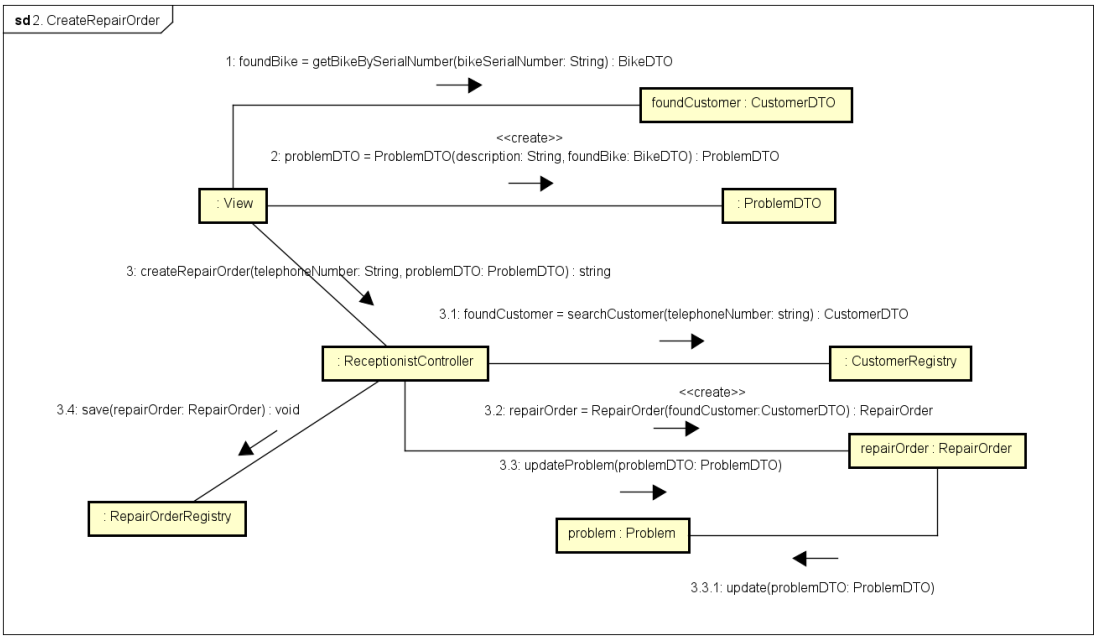

Figure 3: A communication diagram for `createRepairOrder` system operation

3.3. The `findRepairOrder` system operation

The `findRepairOrder` system operation finds the most recent created repair order of a customer. This is done by first fetching all the repair orders filtered by a given telephone number, and the most recent added repair order is selected. The telephone number used to retrieve the repair order comes directly from user input or a previous interaction. The system operation is visualized in Figure 4.

To prevent exposing model internals, `RepairOrderRegistry` returns a `RepairOrderDTO` instead of the repair-order entity itself. The view is never allowed to receive a reference

to a mutable `RepairOrder` object, in order to ensure strict encapsulation and protecting the consistency of the model state.

Cohesion is maintained because each class serves one purpose: the registry looks up stored orders, the entity encapsulates business logic, and the controller only coordinates the sequence of calls. Coupling is kept low because the controller does not know how orders are stored internally.

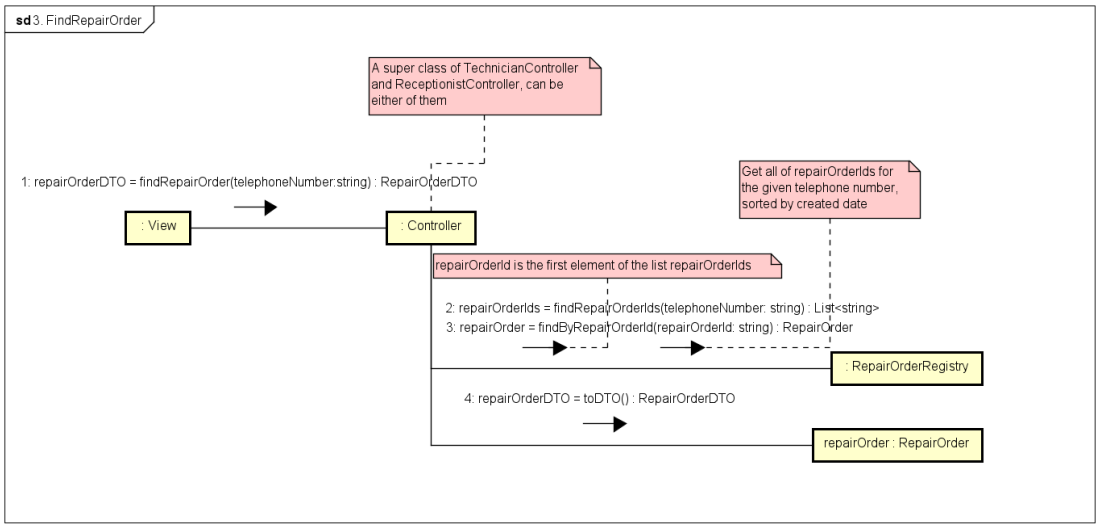

Figure 4: A communication diagram for `findRepairOrder` system operation

3.4. The `addDiagnosticResult` system operation

The `addDiagnosticResult` system operation attaches a diagnostic result onto a repair order. The operation starts by creating a `ResultDTO` object that contains the description as well as `checked` flag for having the diagnostic task checked and `toBeRepaired` flag. Afterwards, the repair order and the diagnostic task are first fetched before updating the diagnostic result of the diagnostic task of the repair order as well as calculating the total cost of repair order and saving it back into its registry. The operation can be visualized in Figure 5.

Diagnostic results are added using a `DiagnosticReportDTO`, which is an immutable data carrier. The actual logic for incorporating the diagnostic report belongs to the `RepairOrder` entity, which updates its internal state through a dedicated method. This ensures that behavior, not the controller, performs the state change, preserving cohesion and encapsulation.

The DTO is never modified. Instead, all updates occur in the model, preventing attribute leakage. Coupling remains low because the controller does not need to know how diagnostic results are stored or validated. All validation and state management sits inside the model layer.

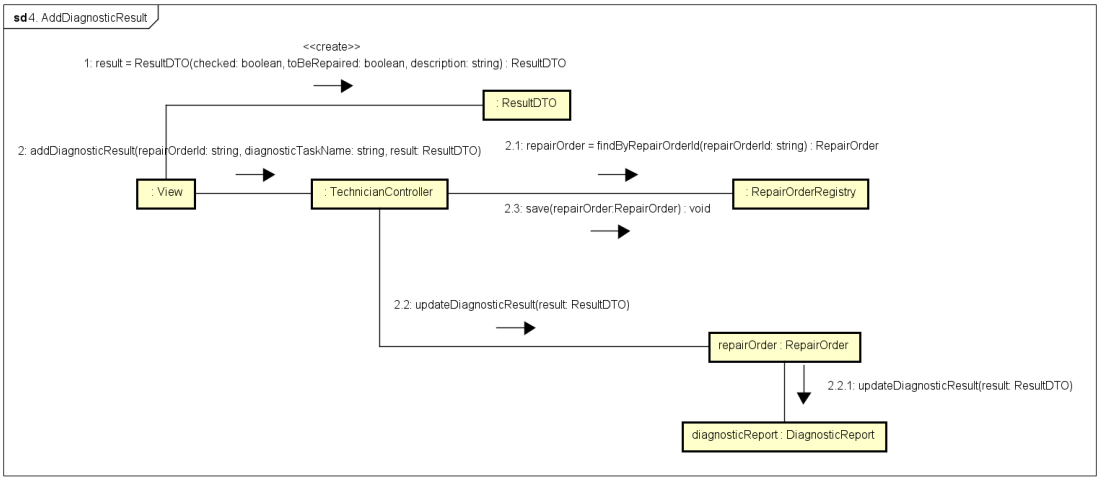

Figure 5: A communication diagram for `addDiagnosticResult` system operation

3.5. The `createProposedRepairTask` system operation

In Figure 6, the `createProposedRepairTask` system operation creates a new proposed repair task for a repair order. First, a `ProposedRepairTaskDTO` is created containing name, description, cost and estimated finish time in days. The DTO object is then attached to a repair order by first fetching the given repair order using a `repairOrderId`, as well as updating the total cost of the repair order. Finally, the operation ends with saving the repair order in its registry.

Each proposed task is encapsulated inside a `ProposedRepairTaskDTO`, which contains immutable data. The task is added to the repair order through a method belonging to the `RepairOrder` entity. This keeps all business behavior inside the model layer and prevents the controller from internal logic responsibilities.

A dedicated `Cost` class is used instead of primitive values. This avoids overuse of primitive values which can increases cohesion by grouping related values and reduces coupling by centralizing cost-related logic in a relevant class. Since the DTO is immutable, objects are kept intact and never hand out modifiable references.

3.6. The `acceptRepairOrder` system operation

The `acceptRepairOrder` system operation accepts a repair order by a repairOrderId, as shown in Figure 7. The `RepairOrder` is first found from a `repairOrderId`, which an `accept` method is called for updating the repair order state to `Accepted`. The repair order is saved back to its registry, as well as printing out the repair order.

Accepting or rejecting a repair order is a state transition and therefore belongs to the `RepairOrder` entity. The controller only calls the corresponding method and does not implement any business logic itself.

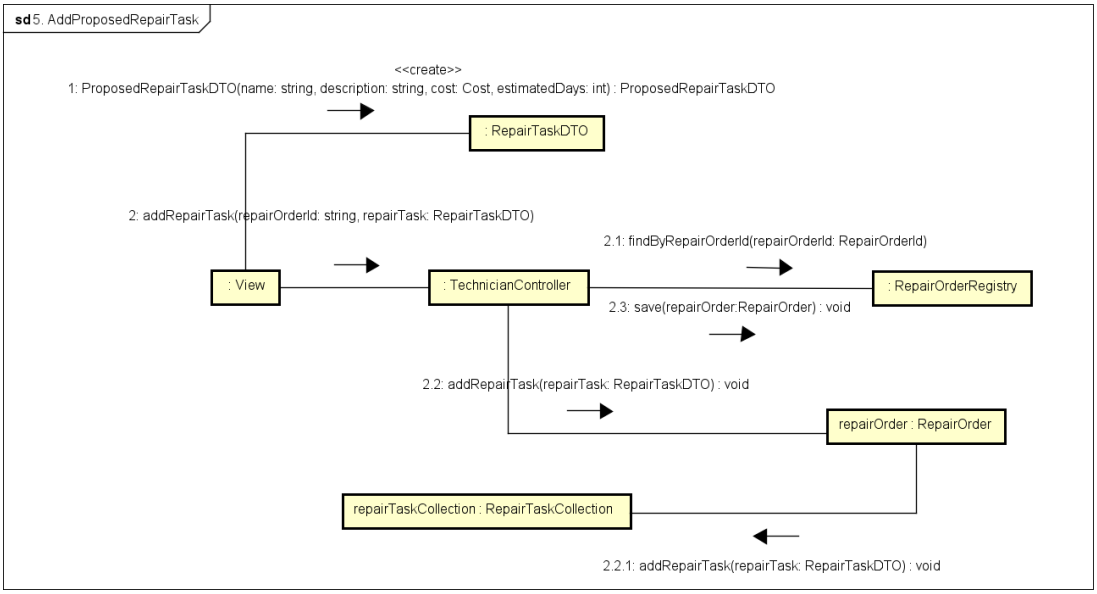

Figure 6: A communication diagram for `createProposedRepairTask` system operation

Encapsulation is achieved by keeping the internal status field of repair order as private. Only well-defined public methods such as `accept()` and `reject()` can modify the state. The object is later returned to view as DTO ensuring that mutable entities are not passed to view. Coupling remains low since no external class needs to know how the state is represented internally.

3.7. The `rejectRepairOrder` system operation

The `rejectRepairOrder` follows the same pattern as `acceptRepairOrder` system operation, but the receipt and reports are not printed out. The action steps for rejecting a repair order is visualized in Figure 8

3.8. Full communication diagram of `Startup`

In startup procedure, given in Figure 9, all entry classes in each layer should be initialized, in order for the program to be initialized. The classes includes creator classes such as `RegistryCreator` for integration layer and `ControllerCreator` for controller layer, as well as `View` which acts as an interactive interface to user. Additionally, `Printer` class which is located in integration layer is also be initialized.

Furthermore, in integration layer, `RegistryCreator` starts up `CustomerRegistry` and `RepairOrderRegistry`, as well as `ControllerCreator` initializes `ReceptionistController` and `TechnicianController`.

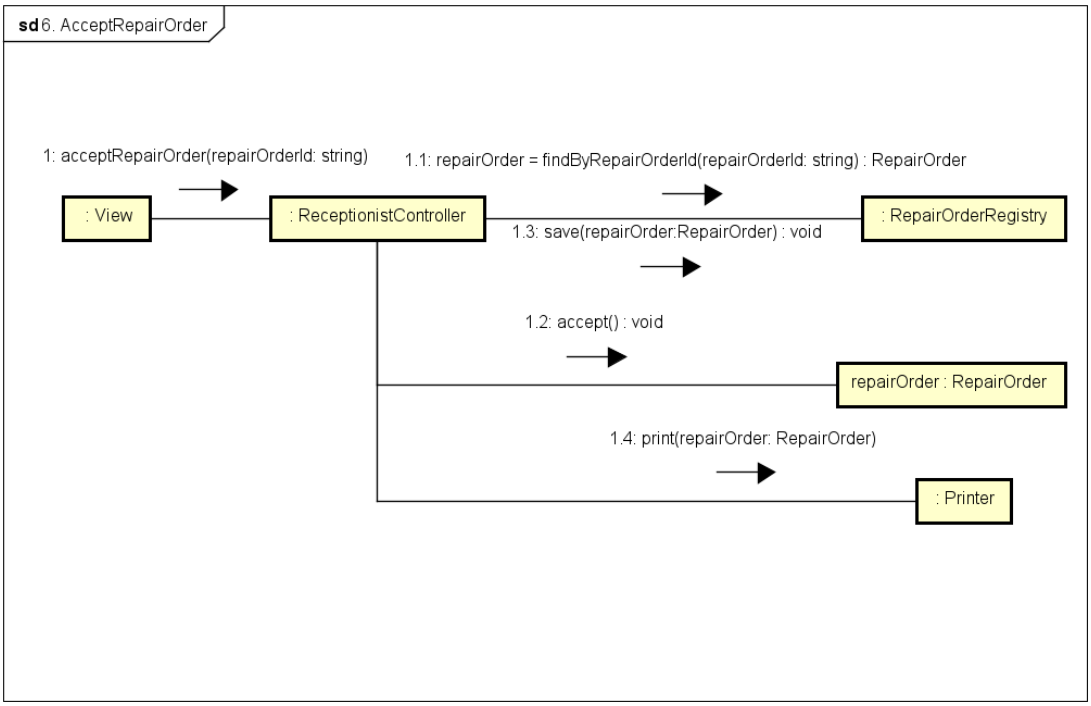

Figure 7: A communication diagram for `acceptRepairOrder` system operation

3.9. Full class diagram

The full class diagram can be visualized in Figure 10, which represents a layered system architecture with five main packages: startup, view, controller, model, and integration. The data package is omitted since the program currently does not store any data into secondary storage, such as disk and database. Each package takes a specific and unambiguous responsibility inside the system, which ensures separation of concerns and maintainability. Additionally, methods with public, protected and default visibility are displayed in the class diagram, while the private ones are hidden, in order to keep the diagram compact with relevant details. Class member fields are omitted from the diagram as well, since all class members are kept private to protect themselves from being corrupted by external factors.

The startup package contains the `Main` class, which serves as the entry point of the application, initializing the necessary class instances.

The view package contains the `View` class, which acts as the user interface layer. Currently, the view class is hardcoded and communicates directly with the control layer by calling system operations as methods in classes of control layer.

The controller package acts as an intermediary control flow layer between the view and model layers. The abstract `Controller` class provides a shared `findRepairOrder`. Two subclasses are created for both `Receptionist` and `Technician`: `ReceptionistController`

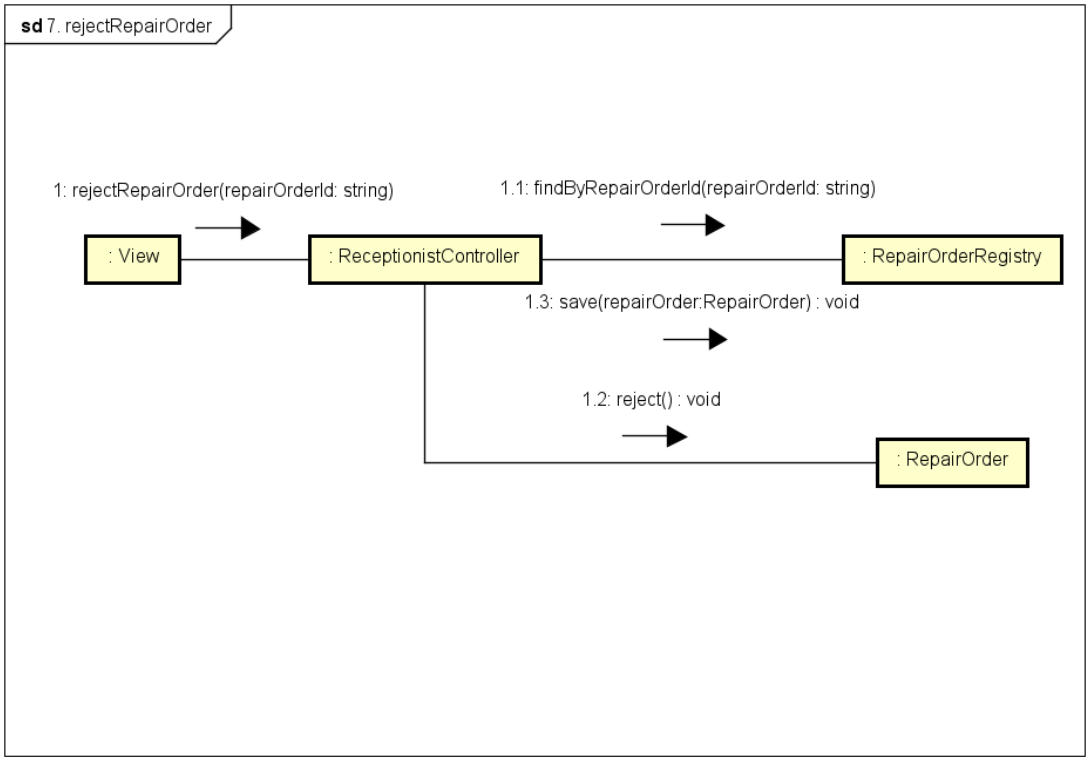

Figure 8: A communication diagram for `rejectRepairOrder` system operation

and `TechnicianController`. The `ReceptionistController` handles operations related to customer interaction, such as searching for customers, creating repair orders, and accepting or rejecting them. The `TechnicianController` manages diagnostic results and proposed repair tasks. Implementing each controller class for each role provides clear separation, while the base class for common logic.

The model package encapsulates the core computational logic. The central class is `RepairOrder`, which maintains repair order informations such as ID, date, cost, and state. It provides methods for updating diagnostic results, adding repair tasks, calculating cost, and changing the order state. Several supporting DTO classes are included, such as `ProblemDTO`, `ProposedRepairTaskDTO`, `DiagnosticReportDTO`, and `DiagnosticTaskDTO`, which serves as data encapsulations. The `Result` class represents the result of each diagnostic task.

The integration package acts as a middle-ground between model and data layer, which caches data from disk and database storage. The classes act as a middle ground, such as `CustomerRegistry` class that stores `CustomerDTO` instances provided with customer data, allowing lookup by phone number as well as `BikeDTO`. The `RepairOrderRegistry` manages storage and retrieval of `RepairOrder` objects. Additionally, `Printer` class is responsible for outputting repair order information.

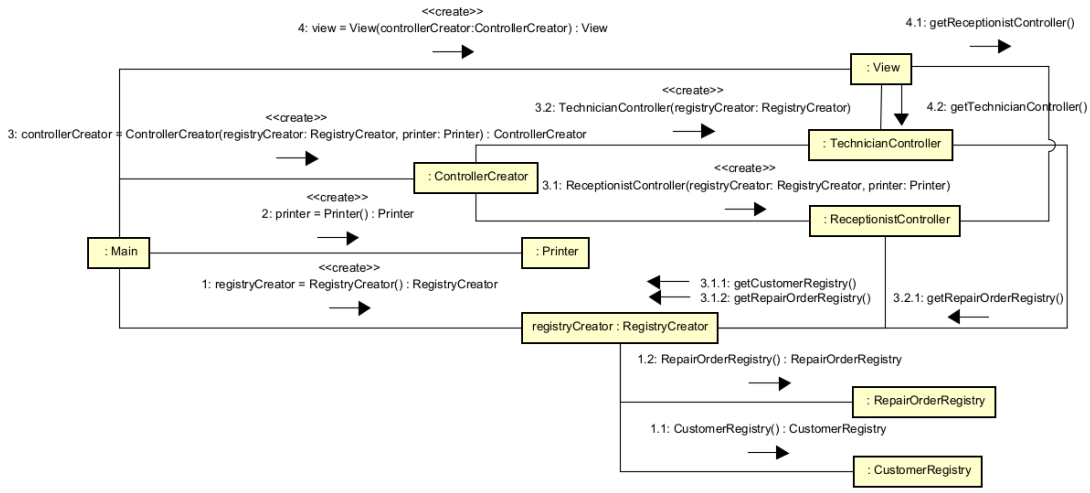

Figure 9: A full communication diagram for startup operation during program initialization. All entry classes of each layer are initialized.

4. Discussion

4.1. Evaluation of MVC Architecture, Layering, and Object-Oriented Design Quality

The design follows the MVC and Layer patterns, by separation into View, Controller, and Model. The View contains only user-interaction logic, which are currently hard-coded, while all system-operation logic is placed in the controllers, and business logic remains in the Model. Additionally, view-related code does not appear in other layer, which confirms that all the concerns are separated into layers that focus on one single, unambiguous responsibility.

Furthermore, the design contains all necessary layers: View, Controller, Model, Integration, and Startup. Multiple controller classes (`ReceptionistController` and `TechnicianController`) shows that there are enough classes to separate responsibilities, and none of the controllers contain core business logic, which stays consistent with the guideline that the controller does not take responsibility in granular and detailed core logic. The Integration layer holds the DTOs and registry classes, ensuring that data access is completely separated from the model logic. This matches the Layer pattern expectations and maintains correct dependency flow.

4.2. Evaluation of high cohesion, low coupling, and good encapsulation

The design produces high cohesion, low coupling, and good encapsulation.

High cohesion High cohesion means that each class has a single, clearly defined responsibility and contains only behavior that belongs to its abstraction. The design achieves this through several structural choices:

- **Entities model one domain abstraction.** The `RepairOrder` entity is cohesive because it encapsulates only the logic associated with the lifecycle of a repair order—its creation, acceptance, rejection, and state transitions. The detailed implementations of updating problems, adding record diagnostic results, and adding repair tasks are intentionally delegated to dedicated classes that handle each detailed responsibility such as `Problem`, `DiagnosticReport`, and `RepairTask`. Additionally, `RepairOrder` does not contain any external responsibilities outside the scope of repair order, such as actions related to printing, telephone formatting, or JSON processing. Instead, these concerns properly belong to other parts of the system.
- **Supporting classes serve only their intended purpose.** The `Printer` class focuses strictly on formatting and printing `RepairOrder` summaries. It does not query registries, compute costs, or change order state, which ensures a clean separation between presentation and domain logic.
- **DTOs does not contain any behavior except representation.** Records such as `CustomerDTO`, `BikeDTO`, and `RepairOrderDTO` are immutable and does not contain any business logic. Their only purpose is to transfer data from the model to the view, while preserving cohesion in both layers.
- **Registries manage only their own data.** `CustomerRegistry` and `RepairOrderRegistry` are responsible only for storing, retrieving, and updating domain objects. However, they do not interpret and take responsibility in high-level business rules, phone number validations, or perform calculations. As a result, registry code remains cohesive and easy to maintain.
- **Utility classes do only one focused job.** The `TelephoneNumber` class is responsible solely for validating and normalizing Swedish telephone numbers. As a result, it does not contain any logic related to customer search, registries, or formatting of any other domain concepts. As phone-number formats are changed, only this class is updated.
- **Value objects encapsulate one concept.** The `Cost` class encapsulates the monetary amount and currency, without mixing in unrelated behavior such as formatting, printing, or persistence. This reduces scattered primitive values that are harder to maintain. Additionally, all operations related to combining costs are placed inside `Cost`, ensuring that no other class repeats the same logic.

Low coupling Low coupling means keeping as few dependencies to other classes and that a program works better when its components rely on each other as little as possible. The design applies this principle in several ways:

- **Controllers never modify entity internals.** Controllers depend only on high-level operations, never on the internal structure of model classes. For example, the `ReceptionistController` never traverses fields of `RepairOrder`. Instead, it calls domain methods such as `accept()` and `reject()` which hiding all internal low-level logic mechanisms. Similarly, the technician does not inspect internal diagnostic task lists but calls `updateDiagnosticResult()`, letting the model to decide how the update is performed.

- **Entities does not know about controllers or view.** The model layer remains independent of higher layers. For instance, `RepairOrder` has no references to `ReceptionistController` or `TechnicianController`, and `DiagnosticReport` does not rely on how the view displays its data. This allows the model with core business logic to change freely. For instance, switching from an `ArrayList` to a priority queue in `RepairTaskCollection` would not affect controllers or the view. The reversed also applies: replacing a simple hardcoded view with a GUI interface view does not either affect the underlying model and its logic.

- **No logic in View.** The `View` only forwards user input and prints results. As a result, it never computes costs, formats telephone numbers, or manipulates repair orders. For example, when the receptionist enters a phone number, the view simply passes it to the controller. All normalization into E.164 format happens in the `TelephoneNumber` class, ensuring the view layer stays thin and decoupled from business rules.

- **RegistryCreator only creates registries.** The `RegistryCreator` is responsible only for constructing registries, but not for maintaining them or performing domain logic. It simply instantiates `CustomerRegistry` and `RepairOrderRegistry` and provides access through getters. This separation allows the model and integration layers to evolve - e.g., adding in-memory cache storages, database connections, or lazy loading proxy objects in the registries - without modifying the controller or startup code.

- **DTOs isolate the model from external dependencies.** Because DTOs such as `CustomerDTO`, `RepairOrderDTO`, and `DiagnosticTaskDTO` are immutable, controllers and views cannot accidentally modify model state. For example, the view may display a `RepairOrderDTO`, but modifying it has no effect on the underlying `RepairOrder` entity. This reduces coupling by preventing accidental back-references or shared mutable data.

- **Utility classes encapsulate cross-cutting logic.** The `TelephoneNumber` class encapsulates all parsing and validation rules for Swedish phone numbers. Neither controllers nor registries need to know how many digits a subscriber number must have or how to handle "00" prefixes. If the formatting rules change, only this utility class needs to be updated, leaving all dependent code untouched.

High encapsulation Encapsulation refers to hiding the internal details of a class by exposing only its public interface, while keeping the private implementation code inaccessible to the rest of the program. And it is also strengthened through the consistent use of immutable DTOs, which prevent the View from mutating internal system state.

- **Private class fields** All of the member variables in the class are kept private. This protects the object invariants from external corruption.
- **Immutable DTOs** `CustomerDTO`, `BikeDTO`, and `RepairOrderDTO` expose only accessors and not mutators ensuring that external layers cannot corrupt the model.
- **Modifications only via entity methods** Model objects also never leak internal references. As an example, `RepairOrder` does not expose its internal detailed functions directly. Instead, all state transitions occur through dedicated methods such as `addRepairTask()`, `accept()`, or `reject()`, which prevents accidental external modification of the class and ensures that all invariants are checked within the model layer.

Putting it together, by using these structural choices, these implentations result in a design where responsibilities are clear, coupling remains minimal, and the system maintains a strong and robust encapsulation boundary.

4.3. Evaluation of Type Correctness, Object Usage, and Data Encapsulation

This section evaluates how the design handles types, object usage, static elements, data integrity, and naming conventions in accordance with the Seminar 2 assessment criteria.

Correct use of parameters, return values, and type clarity All system operations use parameters whose origins are clearly documented. For instance, the `searchCustomer` operation receives a phone number originating from the View, which is then normalized through the `TelephoneNumber` class before being passed to the `CustomerRegistry`. This ensures that values never "appear out of nowhere" and that their origins are always visible in the interaction diagrams. Return values are also clearly defined: model entities are never returned directly to the View; instead, DTOs such as `CustomerDTO` and `RepairOrderDTO` are used, ensuring a well-typed and explicit data flow.

Use of objects instead of primitive data Primitive data is used only when semantically appropriate, for example strings for names and description. In most cases, the design introduces dedicated value objects to avoid primitive obsession. For example, `ProblemDTO` groups the problem description with the bike identifier using `BikeDTO` because both are needed to create a `RepairOrder` while `Cost` encapsulates amount and currency. Using these classes, multiple primitive parameters can be avoided. These choices improve cohesion, reduce duplicated logic, and make method signatures more stable.

Avoidance of static methods and attributes The design generally avoids static methods and fields since they create hidden dependencies, and make software harder to understand, test, and evolve. Static fields are shared across all objects and therefore break the idea that each instance should manage its own internal state. For this reason, instance attributes are preferred in the codebase, for example `ControllerCreator` and `RegistryCreator` storing their configuration as instance fields instead of static variables. However, the only exception with static fields is printout format strings. The reason is that these fields are immutable and intended for globally defined variables which are necessary for `toString` methods in different classes. Additionally, using static field here removes deduplication of string constants in every instance, which reduces unnecessary memory footprint.

Keeping objects intact and protecting internal state Objects are kept intact. DTOs are immutable and expose their internal state only through getters, preventing accidental modification by the View or Controller. Model entities such as `RepairOrder` maintain private fields and update their state exclusively through behavioral methods such as `addTask()` or `setDiagnosticReport()`. Internal collections and entities are never leaked to View layer; instead, the entity provides methods that operate on the data in a controlled manner.

Compliance with Java naming conventions The naming conventions in the design also follows standard Java guidelines. Classes use PascalCase (for example `RepairOrder`, `CustomerRegistry`), methods and fields use camelCase (for example `findRepairOrder()`, `totalCost`), constants use uppercase with underscores if needed while package names are lowercase. This ensures consistency and readability throughout the design.

4.4. Clarity and Completeness of Method, Results, and Discussion

Overall, the report method clearly explains the design by following the OOD approach using MVC and layer patterns, designing one system operation at a time, applying cohesion-coupling-encapsulation guidelines and finally maintaining a class diagrams. These steps are clearly described and presented in a correct order, which makes design process clean and easy to follow.

The result chapter is also clean and explains the outcome of each methodological step, including each system operation. Each system operation design is described using a corresponding interaction diagram and a short explanation of how messages flow across layers. Additionally, in the end of result section, a full class diagram and the startup sequence are laid out, which shows the overall mechanism of the program.

Additionally, a discussion section evaluates the solution against the assessment criteria. It addresses MVC layered architecture, cohesion, coupling, encapsulation, DTO immutability and the separation of concerns across different layers in the architecture. It also includes strengths and limitations, such as the insight that the controller responsibilities are separated by role (ReceptionistController and TechnicianController), which reduces coupling and improving cohesion.

References

Leif Lindbäck, *A First Course in Object-Oriented Development - A Hands-On Approach*, Course material for IV1350, January 14, 2026

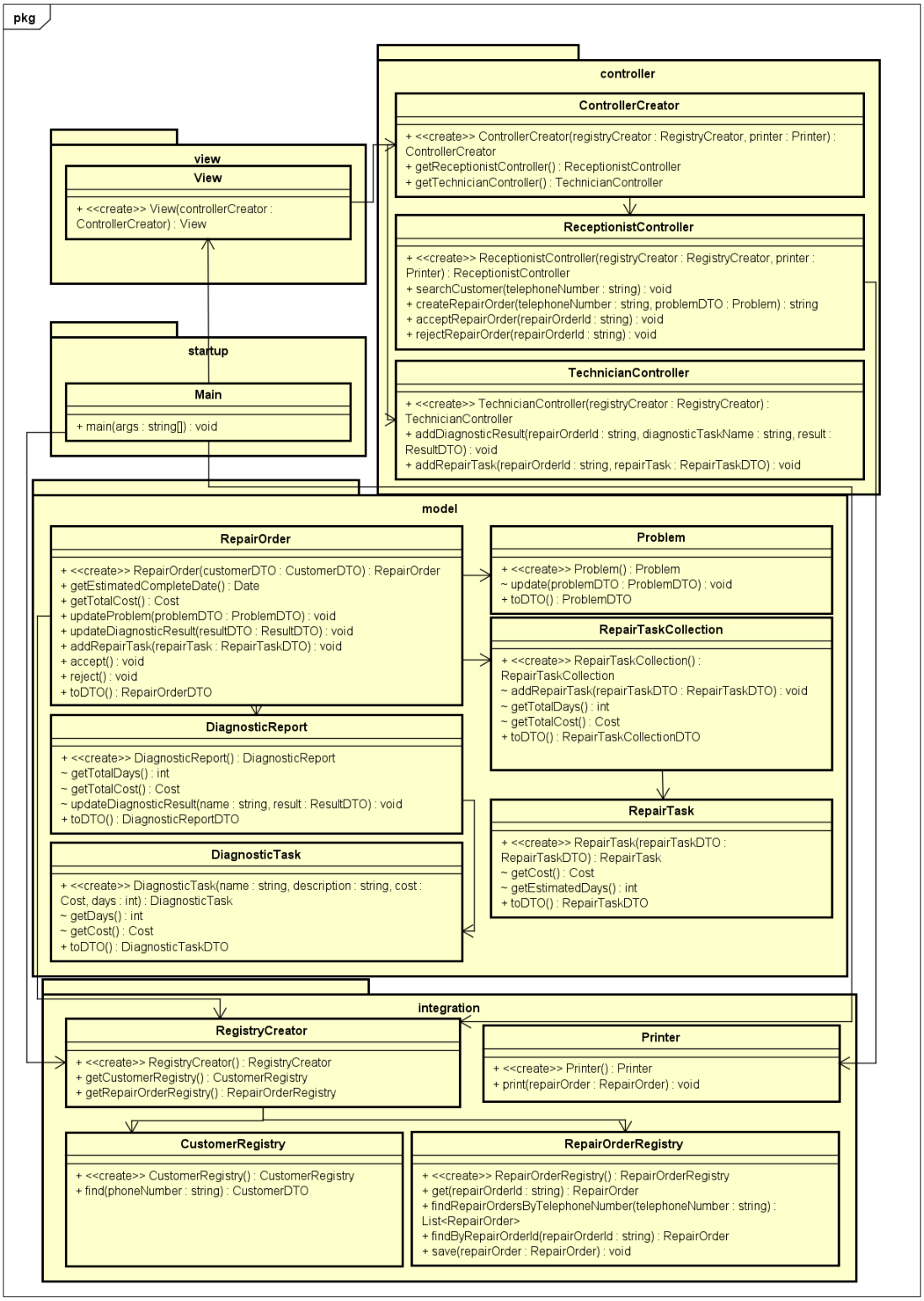

Figure 10: A full class diagram containing the necessary package for each layer. Classes in each layer are visualized as well as their relationships. Class member fields and private methods are not shown in order to make the diagram compact with relevant details.

A. Workflows

In order to design the software, both the domain model (DM) and system sequence diagram (SSD) are required in `Analysis phase`.

The requirement analysis can be stated from the workflows. The basic workflow of the software begins with a customer arriving at the workshop with a broken electric bike. The receptionist identifies the customer using a telephone number and retrieves customer and bike information from the system. Afterwards, the customer provides a problem description of the broken bike, which is created as a repair order by the receptionist and handled by the technician. The technician asks the system for the repair order and performs diagnostic on the bike. Technician concludes the tasks by entering a diagnostic report and proposed repair tasks to the system. After these steps, the receptionist ask customer about accepting or rejecting the repair order. As the customer accepts the order, all repair order informations including estimated complete date are printed out.

One alternative workflow occurs when an invalid telephone number is entered. As the system fails to find the telephone number, the receptionist informs the customer that it doesn't exist in customer registry. Another alternative workflow occur when customer rejects the repair order, which omit the printing of the repair order.

B. Domain model

A domain model for the software system can be visualized in Figure 11, which involves the real-world entities involved in repair process and their relationship. At the center of the domain model is `RepairOrder` which represents a repair case handled by the workshop. A repair order is associated with exactly one `Customer`, who in turn owns more than one `Bike` instances. Each repair order contains a `Problem` where the problem description given by the customer is placed.

During the repair process, a `Technician` performs a diagnostic and produces a `DiagnosticReport`, containing one or more `DiagnosticTask` instances, each describing a specific diagnostic inspection. The outcomes of the diagnostic inspections are written into `Result` entity attached to each `DiagnosticTask` instance. Additionally, the technician also proposes one or more `ProposedRepairTask` with `Cost` instance, which are suggested necessary actions for bike reparation.

The system also includes registry classes that collectively stores all customers and repair orders which is given by `CustomerRegistry` and `RepairOrderRegistry`. A `Workshop` entity employs both `Receptionist` and `Technician` which represent the working roles interacting with the system.

C. System sequence diagrams

The repair electric bike scenario can also be summarized using a system sequence diagram that describe the basic and alternative flows. The basic flow can be visualized in Figure 12.

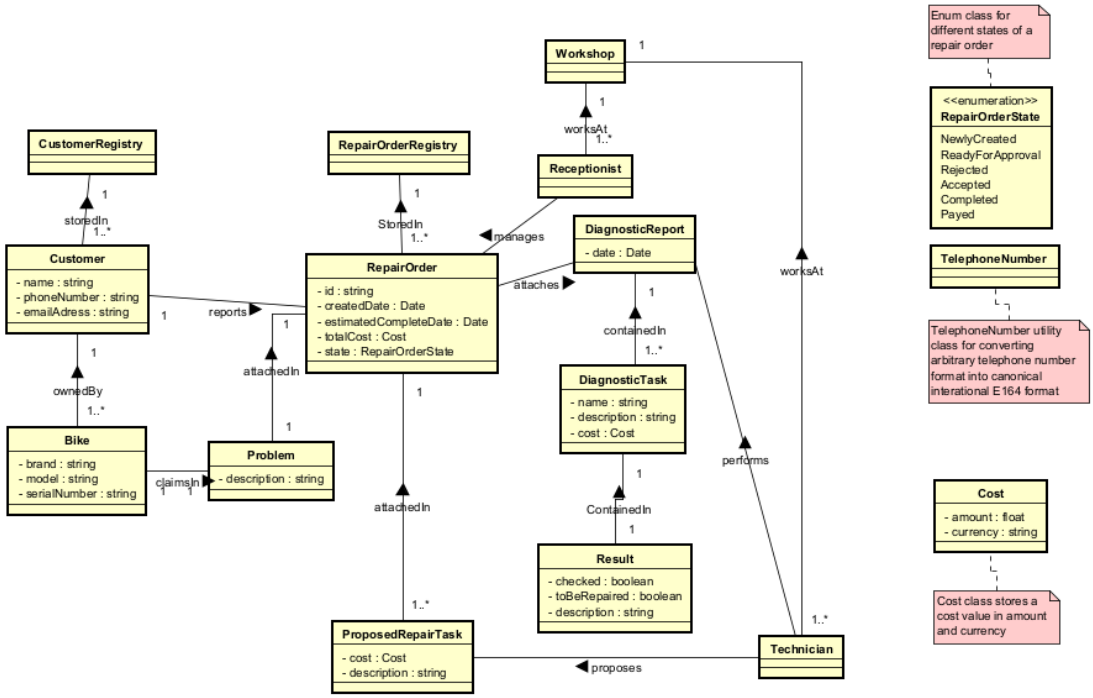

Figure 11: A domain model for the software system is shown. The connections between the entities shows their relationships.

Alternative flows include invalid written telephone number and rejected repair order. Workflow for invalid telephone number can be visualized in Figure 12.

Both the domain model and system sequence diagram provide the analysis foundation for the software design. These diagrams describe the structure of the problem domain and the interactions between external actors and the system, and are therefore used for transforming the requirements specification into an object-oriented design according to the requirements.

Since the requirements includes a layered architecture according to the Model-View-Controller (MVC) pattern as well as applying high coersion, low coupling and good encapsulation, all functionalities described in the requirements specification, including the basic workflow and alternative workflows are supported. Complete interaction diagrams that describe all system operations must therefore also be included, as well as a class diagram that represents all classes involved in the design.

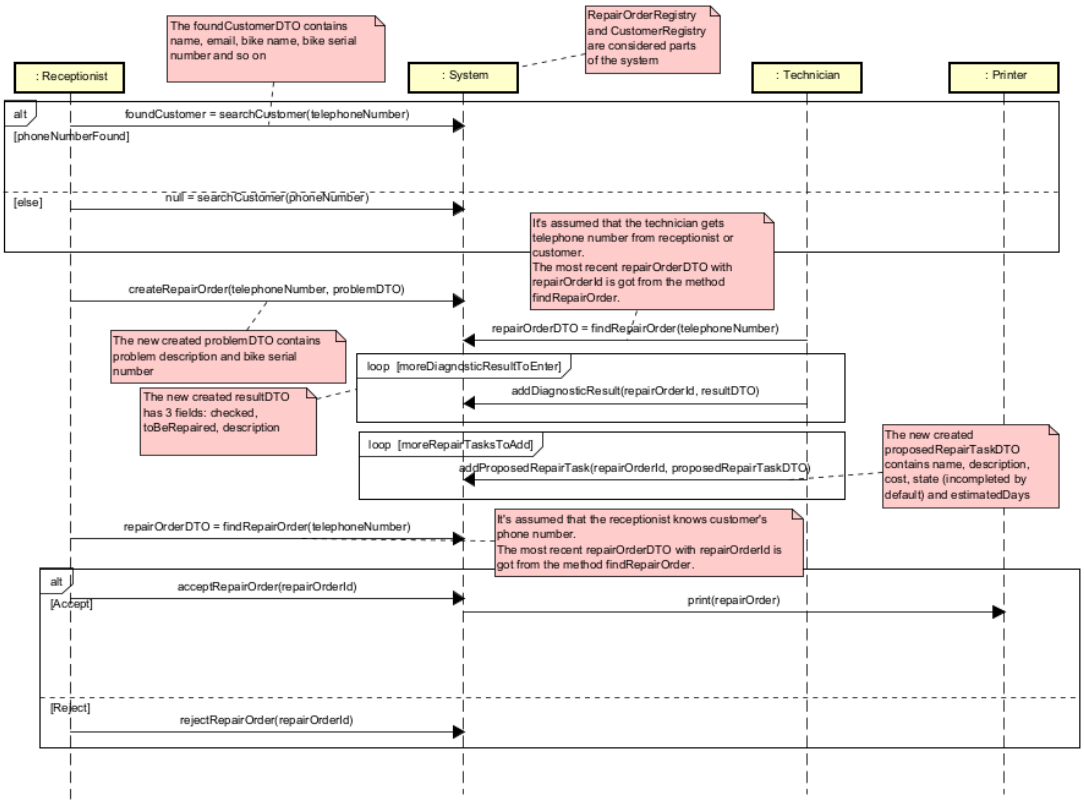

Figure 12: The basic flow, as well as alternative flow is visualized in the system sequence diagram, where each real-world entity interact with the system using system operation. The intent is to enumerate the possible system operations.